REACT BCSL657B (Ability Enhancement Course) 6th Sem CSE

React is a popular, open-source JavaScript library for building user interfaces (UIs), primarily for web applications. It is maintained by Meta (formerly Facebook) and a community of developers.

Why React is Used

React is used to build single-page web applications (SPAs). Allows users to interact with web apps without frequent page reloads. Update only the required part of the page, instead of reloading the entire page when something changes, such as data, state, or user input.

Traditional Websites vs React Websites

Traditional Websites

- Reload the entire page when something changes
- Slower and less smooth user experience

React Websites

- Update only the required part of the page
- No full page reload
- Faster and smoother user experience

Key Reasons for Using React

1. Efficient UI Updates (Reactive Programming)

    - React updates the UI when data or state changes
    - Facebook introduced React to build applications that react to new data instead of refreshing pages
    - This approach is called reactive programming

2. Component-Based Architecture

    - The UI is divided into reusable components (e.g., buttons, forms, headers)
    - Makes code clean, modular, and easy to maintain

3. Fast Performance (Virtual DOM)

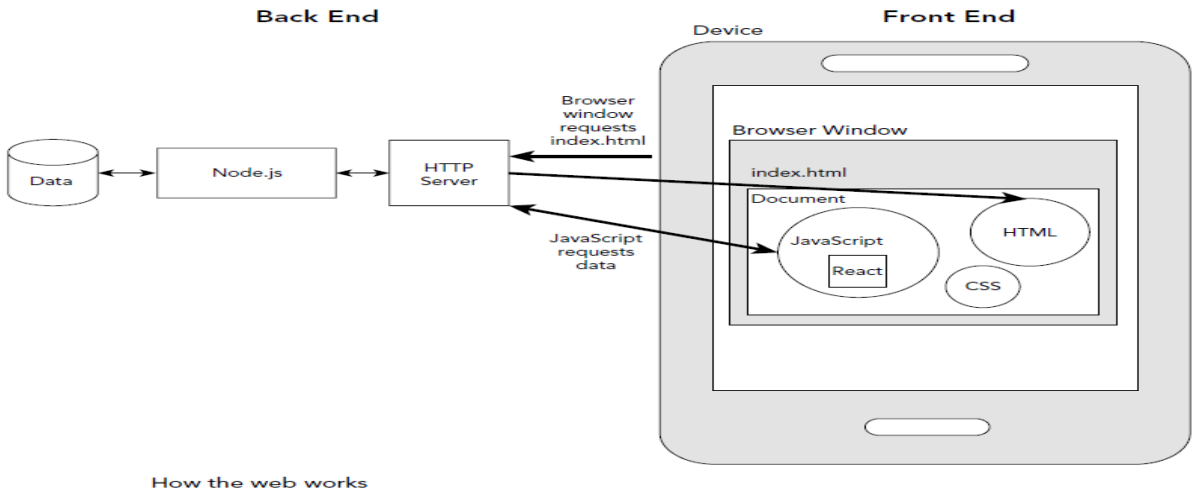
    - React uses a Virtual DOM to track changes
    - Only the necessary parts are updated in the real DOM
    - Improves application performance

Where is React Used?

- Web Applications
- E-commerce platforms: Amazon, Shopify.
- Social media applications: Facebook, Instagram, Gmail, Uber Eats
- Mobile Applications
- Real-Time dashboards, chats.

- WhatsApp Web, Cryptocurrency or stock market dashboards
- Netflix, GitHub

How a React Web App Works



Back End

- **Data**: Stores application data (database).
- **Node.js**: Processes requests and handles application logic.
- **HTTP Server**: Sends and receives requests between the browser and server.

Front End (User Device)

- Runs inside the **browser**.
- The browser first requests **index.html** from the server.
- The document contains:
  HTML – page structure
  CSS – styling
  JavaScript (React) – dynamic behavior and UI updates

Work Flow

1. Browser requests index.html from the server.
2. Server sends the HTML file to the browser.
3. React (JavaScript) runs in the browser.
4. When data is needed, React sends JavaScript requests to the server.
5. Server fetches data from the database and sends it back.
6. React updates only the required part of the UI without reloading the page.

Key Features

1.Component-based architecture.

- React follows a component-based architecture**.**

- The entire screen is broken down into small, independent, reusable parts called components**.** Each component represents a specific portion of the UI**.**
- Examples of components: Header, Navigation bar, Content section, Footer, etc.

Each of these parts is written as a separate functional component**.** It is logical, reusable**,** self-contained piece of code**.** It is a JavaScript function. It returns React elements (usually JSX) that describe what should appear on the screen.

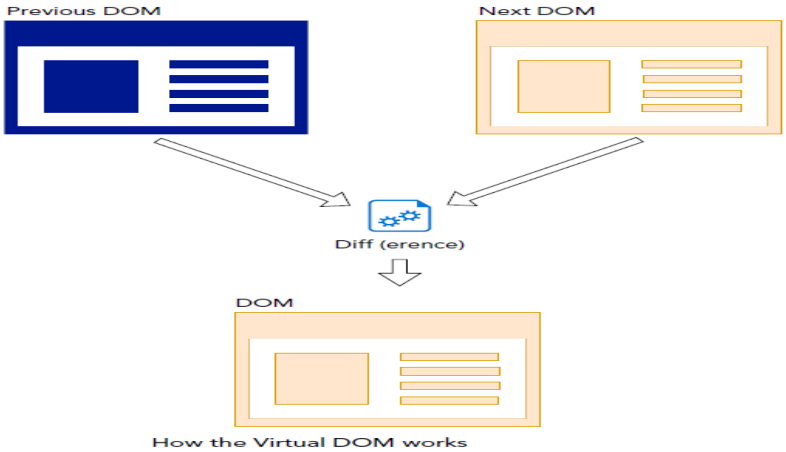
- Example of a Functional Component:

```
function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

Here, Welcome is a functional component. props are inputs passed to the component. The function returns JSX to display UI content.

2. **Virtual DOM:** Efficient updates.

- React uses a Virtual DOM instead of directly updating the real DOM. Only the changed parts are updated.
- The Virtual DOM is a lightweight copy of the real DOM stored in memory.



When a React app loads,

- React creates a Virtual DOM representation of the UI
- When data or state changes, React creates a new Virtual DOM
- React compares the old Virtual DOM with the new one (this process is called diffing)
- React identifies only the changed elements
- Only those changes are applied to the real DOM
- Only modified components are re-rendered , browser remains in the same page.

3.JSX (JavaScript XML) **:**React makes use of JSX, an HTML-in-JavaScript syntax.

- JSX allows writing HTML inside JavaScript
- Example: **const element = <h1>Hello, world!</h1>;**

This is JSX and gets converted into JavaScript behind the scenes.

- Dynamic Content: We can embed JavaScript expressions inside JSX by using curly braces {}:

  const name = “Sunil";
  const element = <h1>Hello, {name}!</h1>;

Instead of separating HTML and JavaScript in different files, JSX allows to write them together, making it easier to build components.

4.React Hooks

Hooks are functions in React that allow functional components to

- Use the state (refers to application data or properties that need to be tracked.)
- update state
- Re-render the UI component without page reload

useState: React Hook

- Most commonly used Hook – useState
- To use the useState Hook, first import it into component.

import { useState } from "react";

- useState accepts an initial state and returns two values:
  The current state.
  A function that updates the state.

Example 1.

const [count, setCount] = useState(0);

Here, **useState(0)** returns two things: A value (state) , **count**
A function **setCount** to change that value
And initializes **count=0**.

Example 2.

```
import { useState } from "react";
function FavoriteColor() {
  const [color, setColor] = useState("red");

  return (
    <>
      <h1>My favorite color is {color}!</h1>
      <button
        type="button"
        onClick={() => setColor("blue")}
      >Blue</button>
    </>
  )
}
```

Hook Rules

- Hooks can only be called inside React function components.
- Hooks can only be called at the top level of a component.

Prerequisites for React:

Install Node.js and npm: Download and Install Node.js

- Go to Node.js official site and download the latest LTS version.
- npm (Node Package Manager) comes with Node.js.
- Verify Installation:
  node -v
  npm -v
  This will display the versions of Node.js and npm installed on system.

Create a React App : Run this command in terminal:

```
npx  create-react-app  my-app
cd  my-app
```

Modify App.js

- Open the Project in **VS Code**
- Open **App.js** inside the **src** folder
- Add the Code

```
import React from 'react';

function Button() {
  return <button>Click Me</button>;
}

function App() {
  return (
    <div>
      <h1>Hello, World!</h1>
      <Button />
    </div>
  );
}
export default App;
```

Run the React App

- In the command prompt, start the development server:
  npm start

- This will open a browser window at http://localhost:3000 showing "Hello, World!" React app with a button.

Modify and See Changes: Any changes we make in App.js or other files in the src folder will automatically update the app in the browser (hot reloading). This code defines a React app with two components: Button and App. The App component renders a "Hello, World!" message and includes the Button component.

(Program explanation)

1. Importing React

```
import React from 'react';
```

- This line imports the React library, which is required to create React components.React provides the tools to define and use components.

2. Defining the Button Component

```
function Button() {

  return <button>Click Me</button>;

}
```

- A functional component named Button.
- This component returns a <button> element with the text "Click Me". When used, it renders this button wherever it is included in the app.

3. Defining the App Component

```
function App() {
  return (
    <div>
      <h1>Hello, World!</h1>
      <Button />
    </div>
  );
}
```

- Another functional component named App. This is the main component of the application.
- It renders a container with:
  A heading (<h1>) that says "Hello, World!".
  The Button component is included here, rendering the button below the heading.

4. Exporting the App Component

```
export default App;
```

- It allows the App component to be the default export of the file. This means that when this file is imported elsewhere, the App component is the one that will be loaded by default.
- It enables this component to be used in other files, such as when rendering the root React component in index.js.

How the Components Work Together

- The App component serves as the parent and the main entry point for the app.
- Inside the App component, the Button component is included using <Button />.
- When App is rendered, it shows: A heading with "Hello, World!". A button below it with the text "Click Me".

Sample1: React App to print welcome

```
import React from 'react';
import './App.css';
function App() {
return (
        <div className="App">
        <h1>Hello, React!</h1>
        <p>Welcome to first React app.</p>
        </div>
);
}
export default App;
```

To create and run app:

```
npx  create-react-app  my-app1
cd  my-app1
npm start
```

Sample2: React App using useState Hook

```
import React, { useState } from 'react';
import './App.css';
function App() {
  const [message, setMessage] = useState("Welcome to React!");
  const handleClick = () => {
    setMessage("You clicked the button!");
  }
  return (
    <div className="App">
      <h1>Hello, React!</h1>
      <p>{message}</p>
      <button onClick={handleClick}>Click Me</button>
    </div>
  );
}
export default App;
```

Styling using CSS: Add styles in src/App.css

App.CSS

```
.App {
   text-align: center;
   margin: 50px;
}

button {
   padding: 10px 20px;
   cursor: pointer;
   margin: 20px;
}

button:hover {
      background-color: #0094ff;
 }
```

Commands:

cd ..

npx create-react-app my-app2

cd my-app2

npm start

Sample 3-React App using useState Hook to demo Dynamic input

```
import React, { useState } from 'react';
import './App.css';

function App() {
   const [name, setName] = useState("");

   const handleChange = (event) => {
      setName(event.target.value);
   }
   return (
      <div className="App">
         <h1>Hello, {name || "Guest"}!</h1>
         <p>Type your name below:</p>
         <input type="text" placeholder="Enter your name" onChange={handleChange} />
      </div>
   );
}
export default App;
```

Styling using CSS: Add styles in src/App.css

App.CSS

```
.App {
   text-align: center;
   margin-top: 60px;
}

input {
   padding: 10px 20px;
}
```

Sample 4- Simple Counter Application in React using useState Hook

```
import React, { useState } from 'react';

import './App.css';

function App() {
   const [count, setCount] = useState(0); // "count" is the state, starting at 0

   return (
      <div className="App">
         <p>Count: {count}</p> {/* Shows the current state */}
         <button onClick={() => setCount(count + 1)}>Increase</button> {/* Updates state */}
      </div>
   );
}
export default App;
```

App.CSS

```
.App {
   text-align: center;
   margin-top: 60px;
}

button {
   padding: 8px 16px;
   font-size: 16px;
   cursor: pointer;

}
```